AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

NAME: RAAGHAV VEL P

CSE-B

CH.SC.U4CSE24155

1. Construct an AVL tree with these numbers:
   157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

   (i) Print the tree showing each level.

   (ii) Check that all nodes are balanced.

Code:

```
//CH.SC.U4CSE24155
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
    int height;
} Node;

// Function to get height
int height(Node *n) {
    if (n == NULL) return 0;
    return n->height;
}

// Max function
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Create new node
Node* newNode(int data) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->data = data;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}
```

```
// Right rotate
Node* rightRotate(Node *y) {
    Node *x = y->left;
    Node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

// Left rotate
Node* leftRotate(Node *x) {
    Node *y = x->right;
    Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

// Get balance factor
int getBalance(Node *n) {
    if (n == NULL) return 0;
    return height(n->left) - height(n->right);
}

// Insert into AVL
Node* insert(Node* node, int key) {
    if (node == NULL)
        return newNode(key);
```

```
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));

    int balance = getBalance(node);

    // LL
    if (balance > 1 && key < node->left->data)
        return rightRotate(node);

    // RR
    if (balance < -1 && key > node->right->data)
        return leftRotate(node);

    // LR
    if (balance > 1 && key > node->left->data) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    // RL
    if (balance < -1 && key < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}
```

```
// Level order traversal
void levelOrder(Node* root) {
    if (root == NULL) return;

    Node* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        int size = rear - front;

        for (int i = 0; i < size; i++) {
            Node* temp = queue[front++];
            printf("%d ", temp->data);

            if (temp->left)
                queue[rear++] = temp->left;
            if (temp->right)
                queue[rear++] = temp->right;
        }
        printf("\n"); // new level
    }
}

// Check AVL balance
int checkAVL(Node* root) {
    if (root == NULL) return 1;

    int balance = getBalance(root);

    if (balance > 1 || balance < -1)
        return 0;

    return checkAVL(root->left) && checkAVL(root->right);
}

// Main
void main() {
    Node* root = NULL;

    int arr[] = {157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117};
    int n = sizeof(arr)/sizeof(arr[0]);

    for (int i = 0; i < n; i++) {
        root = insert(root, arr[i]);
    }

    printf("Level-wise AVL Tree:\n");
    levelOrder(root);

    if (checkAVL(root))
        printf("\nTree is balanced (AVL)\n");
    else
        printf("\nTree is NOT balanced\n");
}
```

Output:

```
C:\II year\4th sem\DAA\LAB\week-5\prog>.\a
Level-wise AVL Tree:
122
111 147
110 112 133 151
117 123 141 149 157

Tree is balanced (AVL)
```

2. Construct a Red-Black tree with the numbers:

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

Print the tree showing **R (red)** or **B (black)** for each node. Check that:

(i) Root is black

(ii) No red node has red children

Code:

```
//CH.SC.U4CSE24155
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    char color; // 'R' or 'B'
    struct Node *left, *right, *parent;
} Node;

Node *root = NULL;

// Create new node
Node* newNode(int data) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->data = data;
    node->color = 'R';
    node->left = node->right = node->parent = NULL;
    return node;
}

// Left rotate
void leftRotate(Node *x) {
    Node *y = x->right;
    x->right = y->left;

    if (y->left != NULL)
        y->left->parent = x;

    y->parent = x->parent;

    if (x->parent == NULL)
        root = y;
    else if (x == x->parent->left)
        x->parent->left = y;
    else
        x->parent->right = y;

    y->left = x;
```

```
    x->parent = y;
}

// Right rotate
void rightRotate(Node *y) {
    Node *x = y->left;
    y->left = x->right;

    if (x->right != NULL)
        x->right->parent = y;

    x->parent = y->parent;

    if (y->parent == NULL)
        root = x;
    else if (y == y->parent->left)
        y->parent->left = x;
    else
        y->parent->right = x;

    x->right = y;
    y->parent = x;
}

// Fix Red-Black violations
void fixInsert(Node *z) {
    while (z->parent != NULL && z->parent->color == 'R') {
        if (z->parent == z->parent->parent->left) {
            Node *y = z->parent->parent->right;

            if (y != NULL && y->color == 'R') {
                z->parent->color = 'B';
                y->color = 'B';
                z->parent->parent->color = 'R';
                z = z->parent->parent;
            } else {
                if (z == z->parent->right) {
                    z = z->parent;
                    leftRotate(z);
```

```
                }
                z->parent->color = 'B';
                z->parent->parent->color = 'R';
                rightRotate(z->parent->parent);
            }
        } else {
            Node *y = z->parent->parent->left;

            if (y != NULL && y->color == 'R') {
                z->parent->color = 'B';
                y->color = 'B';
                z->parent->parent->color = 'R';
                z = z->parent->parent;
            } else {
                if (z == z->parent->left) {
                    z = z->parent;
                    rightRotate(z);
                }
                z->parent->color = 'B';
                z->parent->parent->color = 'R';
                leftRotate(z->parent->parent);
            }
        }
    }
    root->color = 'B';
}

// Insert node
void insert(int data) {
    Node *z = newNode(data);
    Node *y = NULL;
    Node *x = root;

    while (x != NULL) {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
```

```
    }

    z->parent = y;

    if (y == NULL)
        root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;

    fixInsert(z);
}

// Level order print with color
void levelOrder() {
    if (root == NULL) return;

    Node* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        int size = rear - front;

        for (int i = 0; i < size; i++) {
            Node* temp = queue[front++];
            printf("%d(%c) ", temp->data, temp->color);

            if (temp->left)
                queue[rear++] = temp->left;
            if (temp->right)
                queue[rear++] = temp->right;
        }
        printf("\n");
    }
}
```

```c
// Check root black
int isRootBlack() {
    if (root == NULL) return 1;
    return (root->color == 'B');
}

// Check no red-red violation
int checkRedViolation(Node *node) {
    if (node == NULL) return 1;

    if (node->color == 'R') {
        if ((node->left && node->left->color == 'R') ||
            (node->right && node->right->color == 'R'))
            return 0;
    }

    return checkRedViolation(node->left) &&
           checkRedViolation(node->right);
}

// Main
void main() {
    int arr[] = {157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117};
    int n = sizeof(arr)/sizeof(arr[0]);

    for (int i = 0; i < n; i++) {
        insert(arr[i]);
    }

    printf("Level-wise Red-Black Tree:\n");
    levelOrder();

    if (isRootBlack())
        printf("\nRoot is BLACK\n");
    else
        printf("\nRoot is NOT BLACK\n");

    if (checkRedViolation(root))
        printf("No red-red violation\n");
    else
        printf("Red-red violation exists\n");
}
```

Output:

```
C:\II year\4th sem\DAA\LAB\week-5\pro
Level-wise Red-Black Tree:
122(B)
111(R) 147(R)
110(B) 112(B) 133(B) 151(B)
117(R) 123(R) 141(R) 149(R) 157(R)

Root is BLACK
No red-red violation
```